Vibe Coding - JSON 파일 및 파이프라인 레퍼런스

1. JSON 파일 구조

1.1 사용자 계정 — `users.json`

사용자 인증 정보를 저장한다. 비밀번호는 PBKDF2(200,000회 반복)로 해싱되어 `salt:hash` 형태로 보관된다.

```
{
  "users": [
    {
      "username": "admin",
      "password_hash": "salt값:해시값"
    }
  ]
}
```

필드	설명
`username`	로그인 ID
`password_hash`	`salt:hash` 형식의 PBKDF2 해시

> 초기 설정 시 `user.example.json`을 복사하여 사용한다.

1.2 파이프라인 설정 — `pipelines/config.json`

Ollama 서버 연결 정보와 사용할 모델을 지정한다.

```
{
  "ollama_url": "http://localhost:11434",
  "plan_model": "hf.co/CelesteImperia/Gemma-4-26B-MoE-GGUF:IQ3_M",
  "dev_model": "qwen3.5:35b-a3b",
  "single_pass": false,
  "auto_route": true,
  "spec_model_light": "",
  "coder_model_light": ""
}
```

필드	설명
`ollama_url`	Ollama API 서버 주소
`plan_model`	명세(Spec) 생성에 사용하는 모델
`dev_model`	코드 생성에 사용하는 모델

필드	설명
`single_pass`	`true` 건너뛰고 이면 명세 단계를한 번에 코드를 생성
`auto_route`	`true`이면 요청 복잡도에 따라 경량/중량 모델을 자동 선택
`spec_model_light`	간단한 요청에 사용할 경량 명세 모델 (빈 문자열이면 `plan_model` 사용)
`coder_model_light`	간단한 요청에 사용할 경량 코딩 모델 (빈 문자열이면 `dev_model` 사용)

> 초기 설정 시 `config.example.json`을 복사하여 실제 모델명을 입력한다.

1.3 프로젝트 명세

명세 파일은 두 가지 형태로 저장된다.

`specs/{username}/latest.json` — 최신 명세

가장 최근에 생성된 단일 명세서이다. 코드 생성의 입력으로 사용된다.

```json
{
  "username": "aa",
  "chat_id": "mnydaylirjr3p",
  "saved_at": "2026-04-14T17:49:43+09:00",
  "user_message": "원본 사용자 요청 텍스트",
  "spec": {
    "project": {
      "name": "프로젝트명",
      "type": "vanilla-web | react-spa | flutter-app | node-app | cli | data",
      "scope": "frontend | backend | fullstack | cli | data | general",
      "tech_stack": ["HTML", "CSS", "JavaScript"],
      "description": "프로젝트 설명"
    },
    "files": [
      { "path": "index.html", "role": "파일 역할 설명" }
    ],
    "components": [
      {
        "name": "컴포넌트명",
        "props": ["prop1: Type"],
        "behavior": "동작 설명"
      }
    ],
    "api": [],
    "constraints": ["제약 조건"],
    "user_story": "사용자 스토리 요약"
  }
}
```

필드	설명
`username` / `chat_id`	생성자 및 연결된 채팅 세션
`saved_at`	명세 저장 시각 (ISO 8601)
`user_message`	원본 사용자 요청
`spec.project`	프로젝트 메타 정보 (이름, 유형, 범위, 기술 스택)
`spec.files`	생성할 파일 목록과 각 파일의 역할
`spec.components`	UI 컴포넌트 정의 (이름, props, 동작)
`spec.constraints`	구현 시 지켜야 할 제약 조건
`spec.user_story`	사용자 관점의 기능 요약

`specs/{username}/{YYYYMMDD}.jsonl` — 날짜별 명세 이력

해당 날짜에 생성된 모든 명세를 JSONL(한 줄에 하나의 JSON) 형식으로 누적 저장한다.

```
{"saved_at":"2026-04-14T14:49:59+09:00","chat_id":"mny7beok5o58y","user_message":"도시 정보 웹...","spec":{...}}
{"saved_at":"2026-04-14T14:59:04+09:00","chat_id":"mny7nad3ydo16","user_message":"todo리스트를...","spec":{...}}
{"saved_at":"2026-04-14T17:49:43+09:00","chat_id":"mnydaylirjr3p","user_message":"음악 추천기...","spec":{...}}
```

1.4 채팅 로그

채팅 로그도 두 가지 형태로 저장된다.

`chat_logs/{username}/state.json` — UI 상태

채팅 히스토리, 메시지, UI 설정을 저장한다. 프론트엔드가 직접 읽고 쓴다.

```
{
  "chats": [
    {
      "id": "채팅ID",
      "title": "채팅 제목",
      "messages": [
        {
          "id": "메시지ID",
          "requestId": "요청ID",
          "role": "user | assistant",
```

```
            "content": "메시지 내용"
          }
        ],
        "systemPrompt": "",
        "model": "dual-model",
        "pinned": false,
        "createdAt": 1776229896493,
        "updatedAt": 1776229909902
      }
    ],
    "settings": {
      "theme": "system | light | dark",
      "fontSize": 14,
      "temperature": 0.7,
      "runMode": "confirm | instant"
    }
  }
```

필드	설명
`chats[].messages`	대화 내역 (role: user/assistant)
`chats[].model`	사용 모델 (`dual-model` 등)
`settings.theme`	UI 테마
`settings.runMode`	`confirm`: 명세 확인 후 코드 생성, `instant`: 즉시 생성

`chat_logs/{username}/log.jsonl` — 요청/응답 로그

모든 요청과 응답을 JSONL 형식으로 시간순 기록한다. 디버깅 및 이력 추적용이다.

```
{"date":"2026-04-15","time":"14:42:30","input":"다크 모드를 지원하는 대시보드 만들어줘","response":"<details><summary>..."}
{"date":"2026-04-15","time":"14:42:30","input":"현제 코드는 프론트만 있는데 백엔드도 추가해줘","response":"네, 백엔드 기능을..."}
```

필드	설명
`date` / `time`	요청 일시
`input`	사용자 입력 메시지
`response`	어시스턴트 전체 응답 (명세서 + 코드 포함)

---

2. 파이프라인 (`pipelines/dual_model_pipeline.py`)

2.1 요청 라우팅

사용자 메시지가 들어오면 먼저 의도를 분류한다.

```
사용자 메시지 → _route_request()
                  ├─ "chat"           : 일반 대화 (코딩 무관)
                  ├─ "simple_code"    : 간단한 코딩 요청
                  └─ "full_spec_code" : 복잡한 코딩 요청
```

분류 방식:

- **키워드 기반**: "만들어", "구현해", "짜줘" 등 동작 동사 패턴과 기술 키워드로 1차 판별
- **LLM 기반**: `auto_route=true`일 때 라우터 모델이 intent와 complexity를 JSON으로 반환

2.2 메인 처리 흐름

```
┌──────────────────────────────────────────────────────────┐
│                      사용자 메시지                        │
└──────────────────────────┬───────────────────────────────┘
                           ▼
                  ┌── 라우팅 분류 ──┐
                  │                 │
        ┌─────────┼─────────┐       │
        ▼         ▼         ▼       │
     chat    simple    full_spec    │
       │      _code      _code      │
       ▼        │         │         │
   Chat 모델    │         │         │
   직접 응답    │         │         │
                ▼         ▼         │
            ┌─────────────────┐     │
            │  single_pass?   │     │
            │  true    false  │     │
            └──┬──────────┬───┘     │
               ▼          ▼         │
         Coder 모델  Plan 모델      │
         한 번에     (Spec JSON     │
         코드 생성    생성)          │
                       │           │
                       ▼           │
                  Spec 검증 & 저장  │
                       │           │
                       ▼           │
                  Coder 모델       │
                  (Spec → 코드     │
                   스트리밍 생성)   │
                       │           │
└──────────────────────┴───────────┘
                       ▼
                 코드 스트리밍 응답
```

2.3 듀얼 모델 파이프라인 (핵심 흐름)

`single_pass=false`일 때 두 모델이 순차적으로 동작한다.

1단계 — Plan Model (명세 생성)

- 사용자 요청을 분석하여 JSON 형식의 기술 명세서를 생성
- 프론트엔드 프레임워크 선택 규칙 적용 (Vanilla HTML / React / Flutter)
- 생성된 명세는 `specs/{username}/latest.json`에 저장
- 완료 후 `keep_alive=0`으로 모델을 VRAM에서 해제

2단계 — Coder Model (코드 생성)

- 명세서 JSON을 입력받아 파일별 완성 코드를 스트리밍 생성
- 출력 형식: ```파일경로 블록으로 파일 구분

2.4 수정(Diff) 파이프라인

기존 코드를 수정할 때 사용하는 `pipe_modify()` 흐름이다.

```
수정 요청 + 원본 Spec + 기존 파일들
        │
        ▼
   Plan Model → Diff Spec JSON 생성
   (변경할 파일, 새 제약조건, 삭제할 컴포넌트)
        │
        ▼
   Coder Model → 변경된 파일만 코드 생성
```

Diff Spec 구조:

```
{
  "modified_files": [
    { "path": "style.css", "changes": "버튼 색상 변경" }
  ],
  "new_constraints": ["hover 애니메이션 추가"],
  "removed_components": []
}
```

2.5 외부 데이터 페칭

사용자 요청에 실시간 정보가 필요하면 외부 데이터를 수집한다.

소스	함수	용도
Google 검색	`_google_search()`	범용 웹 검색 (1순위)

2.6 모델 선택 전략

`auto_route=true`일 때 요청 복잡도에 따라 모델을 분기한다.

복잡도	Spec 모델	Coder 모델
simple	`spec_model_light` (설정 시)	`coder_model_light` (설정 시)
complex	`plan_model`	`dev_model`

경량 모델이 설정되지 않으면 기본 모델(`plan_model` / `dev_model`)을 사용한다.